Credit Card Approval Prediction

(by Using Machine Learning)

Project Description:

Credit Card Approval Prediction Using Machine Learning is an AI-powered web application that automates the credit card approval process by predicting whether an applicant is eligible for a credit card based on their financial and personal information. The system uses Machine Learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and XGBoost to analyse historical applicant data and generate accurate approval predictions. The best-performing model is deployed using Flask, enabling users to receive real-time approval or rejection results through a simple and user-friendly interface. By reducing manual effort and improving decision-making speed, the system helps financial institutions process applications more efficiently and consistently.

Problem Statement

Manual credit card approval processes are inconsistent, slow, and prone to subjective bias, which can result in creditworthy applicants being rejected or high-risk applicants being approved. There is a need for a data-driven, automated system that can accurately classify credit card applications as approved or rejected based on applicant attributes, thereby improving decision speed, consistency, and reliability for financial institutions.

Scenarios:

Scenario 1: Standard Credit Card Application

A customer applies for a credit card by entering their employment details, annual income, education level, housing status, and family information through the web application. The system preprocesses the input data and uses the trained Machine Learning model to predict whether the application should be **Approved** or **Rejected**, displaying the result instantly.

Scenario 2: High-Risk Applicant Evaluation

A bank officer evaluates an applicant with an unstable employment history and relatively low annual income. After submitting the applicant's details, the system analyses multiple financial and demographic factors using the trained model and predicts a **Rejected** status, helping the bank minimize potential financial risk and make informed lending decisions.

Scenario 3: Bulk Customer Screening

A financial institution processes numerous credit card applications daily. Bank employees enter each applicant's information into the system, which rapidly evaluates every application using the deployed Machine Learning model. The application provides immediate approval

predictions, significantly reducing manual verification time, improving operational efficiency, and maintaining consistent decision-making across all applications.

Objectives

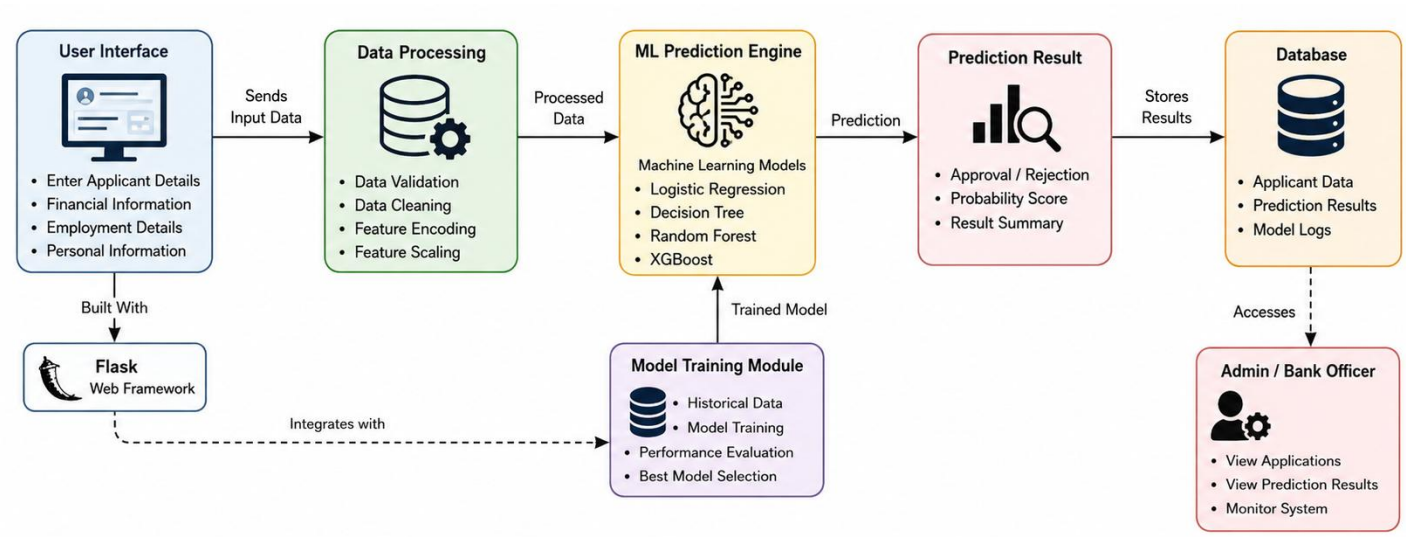
- To analyse and preprocess a real-world credit card application dataset (clean_dataset.csv) for use in machine learning models.
- To engineer meaningful features, such as the Debt-to-Income Ratio, that improve predictive performance.
- To train and compare multiple classification algorithms — Logistic Regression, Decision Tree, Random Forest, and XGBoost — for predicting credit card approval outcomes.
- To evaluate model performance using standard classification metrics including Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC.
- To deploy the best-performing model as an accessible web application for real-time predictions.

Scope

The scope of this project is limited to binary classification of credit card applications (Approved / Rejected) using the provided dataset. The system is intended as an academic demonstration of an end to-end machine learning workflow — from data preprocessing to deployment — and is not intended for production use in an actual financial institution without further regulatory, fairness, and security review.

Technical Architecture:

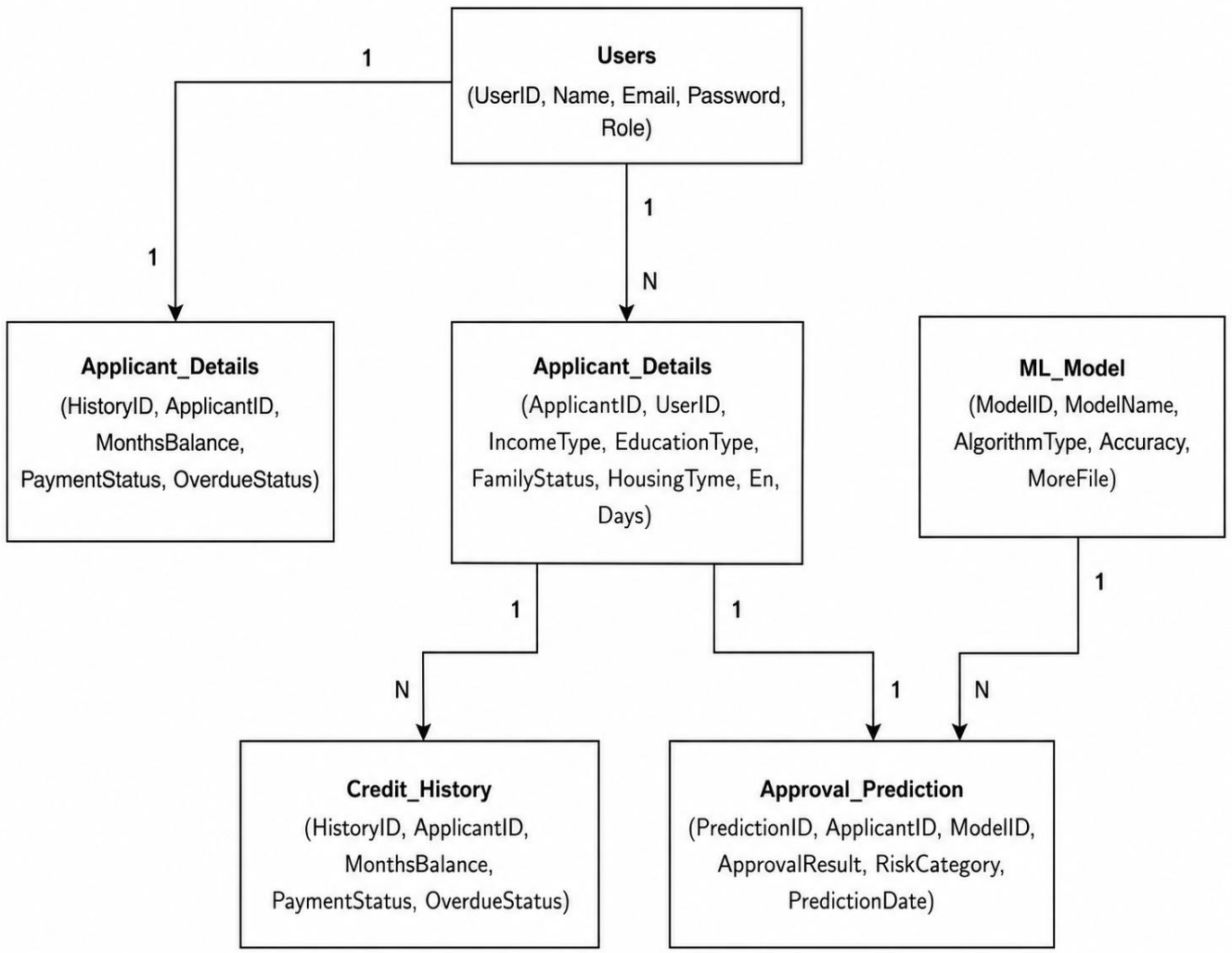
Architecture Flow



ER Diagram

Description:

The **Credit Card Approval Prediction System** follows a modular machine learning architecture to provide fast and accurate credit card approval decisions. Users enter applicant information through a **Flask-based web application**, where the input data is validated, cleaned, encoded, and pre-processed before being sent to the **Machine Learning Prediction Engine**. The system evaluates the application using trained models such as **Logistic Regression, Decision Tree, Random Forest, and XGBoost**, with the best-performing model selected for real-time predictions. The prediction result (Approved or Rejected) is displayed instantly and stored for future reference and analysis. This architecture enables financial institutions to automate the credit card approval process, improve decision-making accuracy, reduce manual effort, and deliver a seamless user experience.



Pre-requisites

- Python Programming Proficiency (Python 3.x)
- Machine Learning Fundamentals — scikit-learn, XGBoost
- Data Handling Libraries — Pandas, NumPy
- Data Visualization — Matplotlib / Seaborn (for EDA)
- Web Framework Knowledge — Flask, for deployment of the prediction interface
- Version Control with Git

Project Workflow:

Milestone 1: Data Collection and Understanding :

- Activity 1.1: Import the dataset (clean_dataset.csv) and inspect its structure, columns, and data types.
- Activity 1.2: Perform exploratory data analysis (EDA) to understand feature distributions and class balance between Approved and Rejected applications.
- Activity 1.3: Identify categorical and numerical features relevant to the prediction task.

Milestone 2: Data Preprocessing and Feature Engineering

- Activity 2.1: Verify the dataset for missing values and handle them appropriately.
- Activity 2.2: Check for and remove duplicate records to ensure data quality.
- Activity 2.3: Apply one-hot encoding to categorical variables to convert them into a numerical format suitable for machine learning models.
- Activity 2.4: Apply a log transformation to the Income feature to reduce skewness and stabilize variance.
- Activity 2.5: Engineer a new Debt-to-Income Ratio feature to better capture applicant financial risk.
- Activity 2.6: Split the dataset into training and testing sets.
- Activity 2.7: Apply StandardScaler to numerical features where appropriate, to normalize feature ranges for scale-sensitive algorithms.

Milestone 3: Model Building

- Activity 3.1: Train a Logistic Regression model as a baseline classifier.
- Activity 3.2: Train a Decision Tree classifier to capture non-linear decision boundaries.
- Activity 3.3: Train a Random Forest ensemble model to improve generalization and reduce overfitting.
- Activity 3.4: Train an XGBoost model to leverage gradient boosting for potentially higher predictive accuracy.

Milestone 4: Model Evaluation and Selection

- Activity 4.1: Evaluate each trained model on the test set using Accuracy, Precision, Recall, and F1-Score.
- Activity 4.2: Generate a Confusion Matrix for each model to analyse true/false positive and negative predictions.
- Activity 4.3: Compute the ROC-AUC score to assess the models' ability to discriminate between classes.
- Activity 4.4: Compare all models and select the best-performing algorithm for deployment. Based on the evaluation results (detailed in Section 5 — Results), the XGBoost Classifier achieved the highest accuracy of 89.86% and was selected as the final model for deployment.

Milestone 5: Deployment

- Activity 5.1: Save the best-performing trained model (XGBoost) using joblib for reuse in the web application.
- Activity 5.2: Build a Flask-based web application with an HTML/CSS front-end that collects applicant details through an application form.
- Activity 5.3: Integrate the trained model with the Flask backend (app.py) to generate real time approval predictions via a /predict API endpoint.
- Activity 5.4: Deploy the application on Render, making it publicly accessible as a live web service.
- Activity 5.5: Test the live application end-to-end to verify correct functioning, from form submission to prediction display.

The application has been successfully deployed and is live at the following URL:

https://ai-ml-smartbridge-project-credit-card.onrender.com

Full implementation details, source code, and screenshots of the deployed web application are provided in Section 6 (Web Application).

Methodology

Dataset Description

Attribute	Details
Dataset Name	clean_dataset.csv
Type	Structed
Target Variable	Credit Card Approval Status(accepted/rejected)
Feature Types	Categorical & Numerical attributes of attributes

Data Preprocessing Steps

- Data Loading and Inspection: The dataset (690 records, 16 columns) was loaded using Pandas, followed by inspection using df.head(), df.tail(), df.info(), and df.describe() to understand structure, data types, and summary statistics.
- Missing Value Verification: The dataset was checked for missing/null values prior to model training to ensure data completeness.
- Duplicate Checking: Duplicate records were identified and handled to prevent data leakage and bias.
- Categorical Encoding: Categorical features (Industry, Ethnicity, Citizen) were encoded using Label Encoding / One-Hot Encoding to convert them into a numerical format suitable for machine learning models.
- Log Transformation: The Income feature was log-transformed to reduce right-skewness caused by extreme high-income values (up to ₹100,000).
- Feature Engineering: A Debt-to-Income Ratio feature was derived to capture applicant financial risk more effectively.
- Train-Test Split: The dataset was divided into training (80%) and testing (20%) subsets using train_test_split from scikit-learn to evaluate model generalization.
- Feature Scaling: StandardScaler was applied to numerical features where required by the algorithm (e.g., Logistic Regression).

Algorithms Used

- Logistic Regression
- Decision Tree Captures
- Random Forest
- XGBoost

Evaluation Metrics

- Accuracy — Overall proportion of correctly classified applications.
- Precision — Proportion of predicted approvals that were actually correct.
- Recall — Proportion of actual approvals that were correctly identified.
- F1-Score — Harmonic mean of Precision and Recall, useful for imbalanced classes.
- Confusion Matrix — Detailed breakdown of true/false positives and negatives
- ROC-AUC — Measures the model's ability to distinguish between approved and rejected classes across thresholds

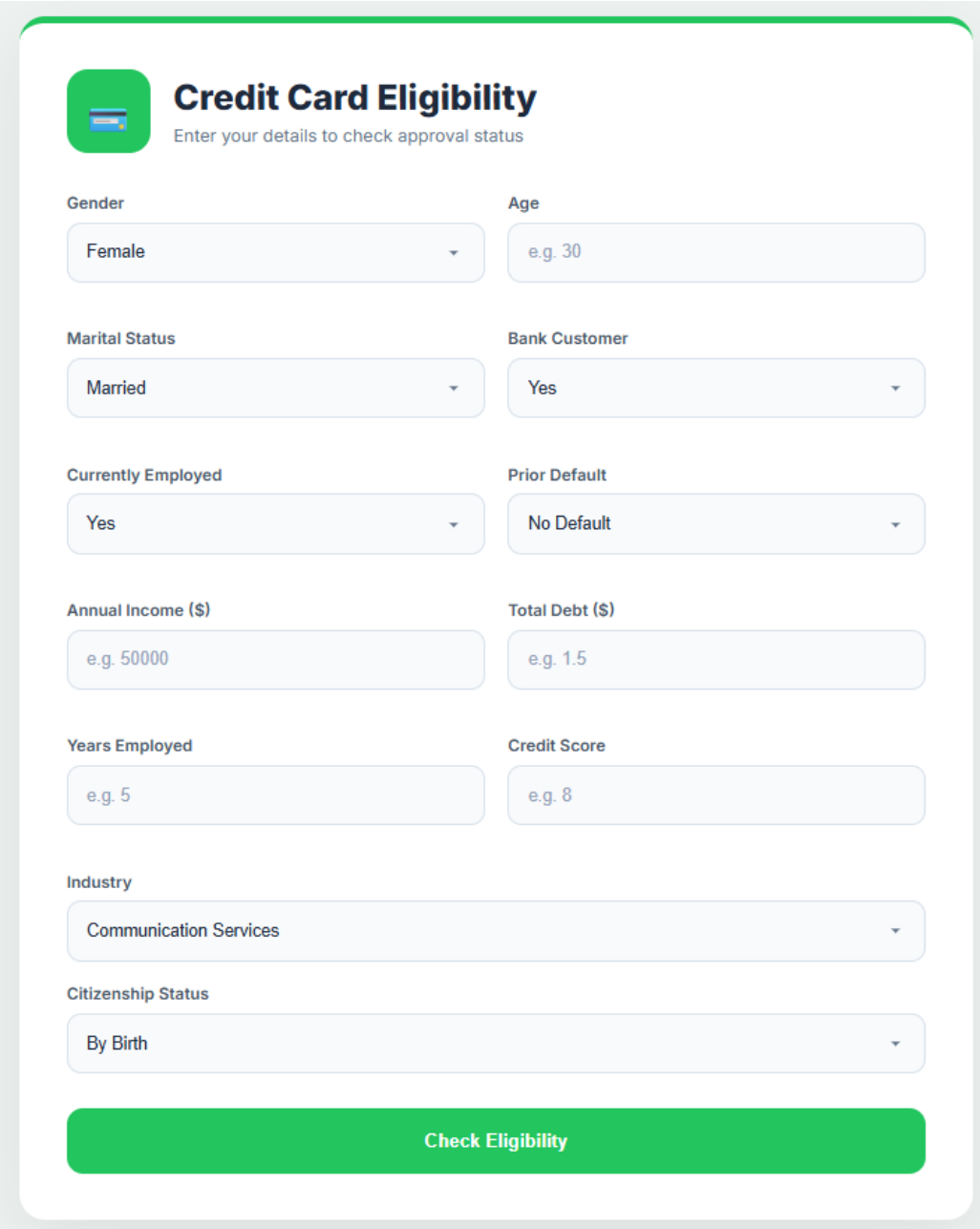
Web Application

The trained XGBoost model has been deployed as an interactive web application named "Credit Card Approval Prediction", built using Flask (backend) and HTML/CSS/JavaScript (frontend). The application allows a user to enter applicant details through a form and receive an instant Approved / Not Approved prediction along with a confidence percentage.

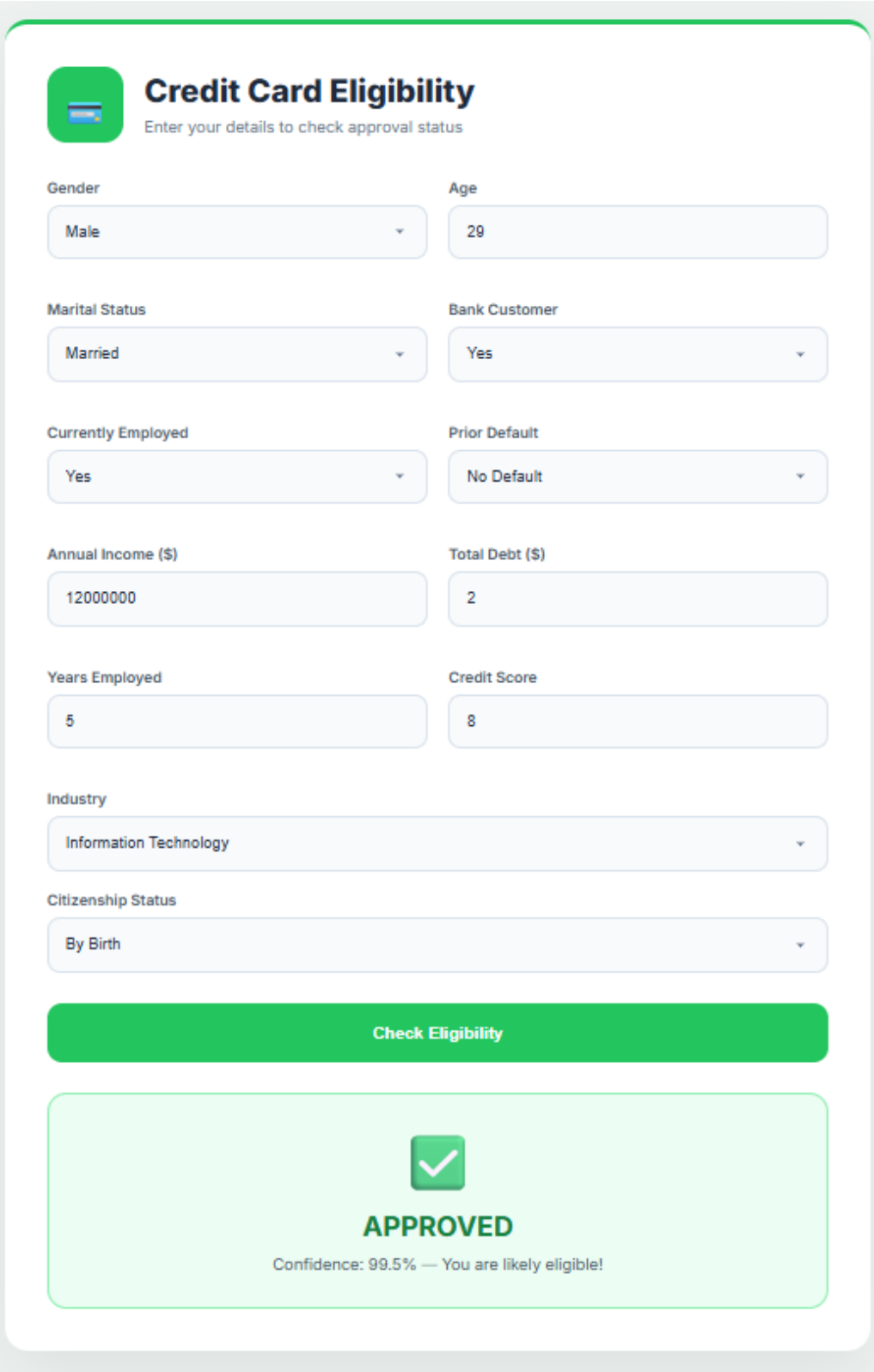
The application has been successfully deployed and is publicly accessible at:

https://ai-ml-smartbridge-project-credit-card.onrender.com

Application Form



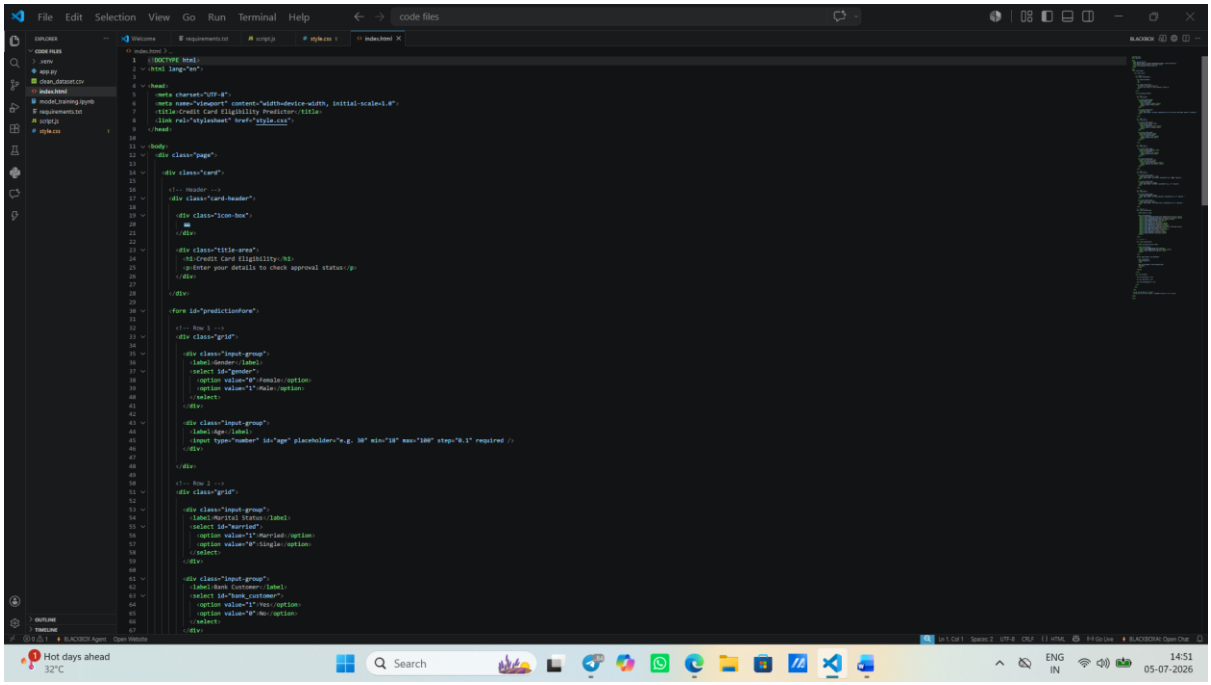
Prediction of Result

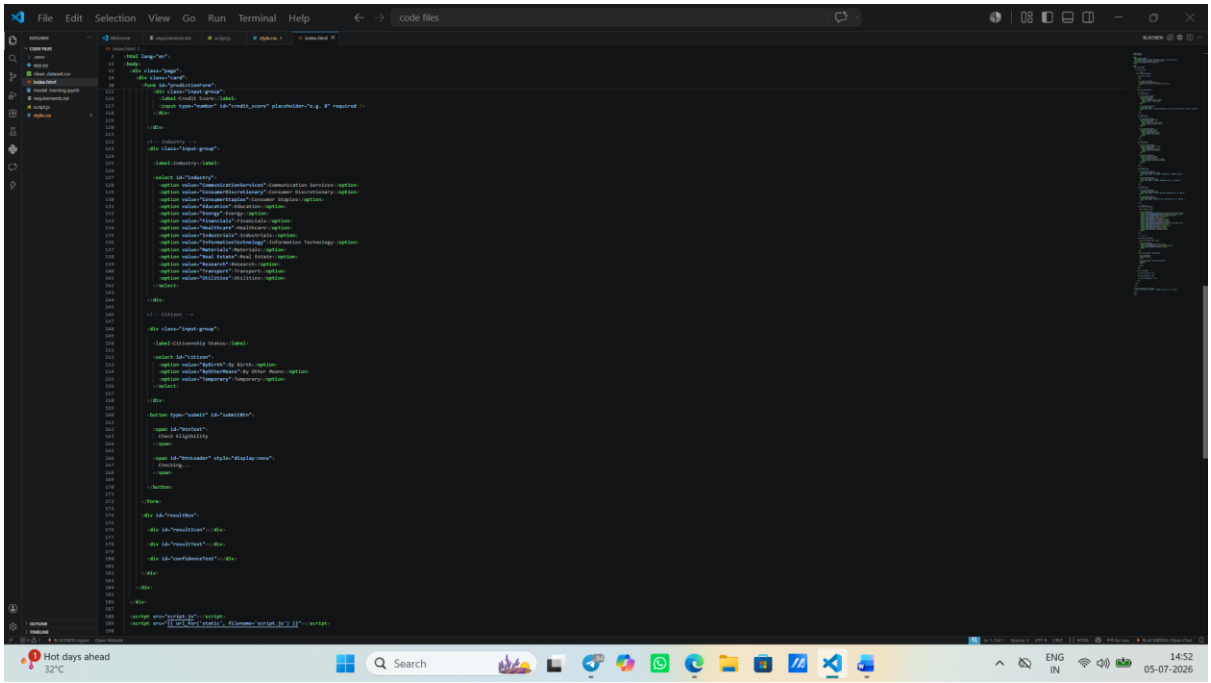


Frontend Implementation

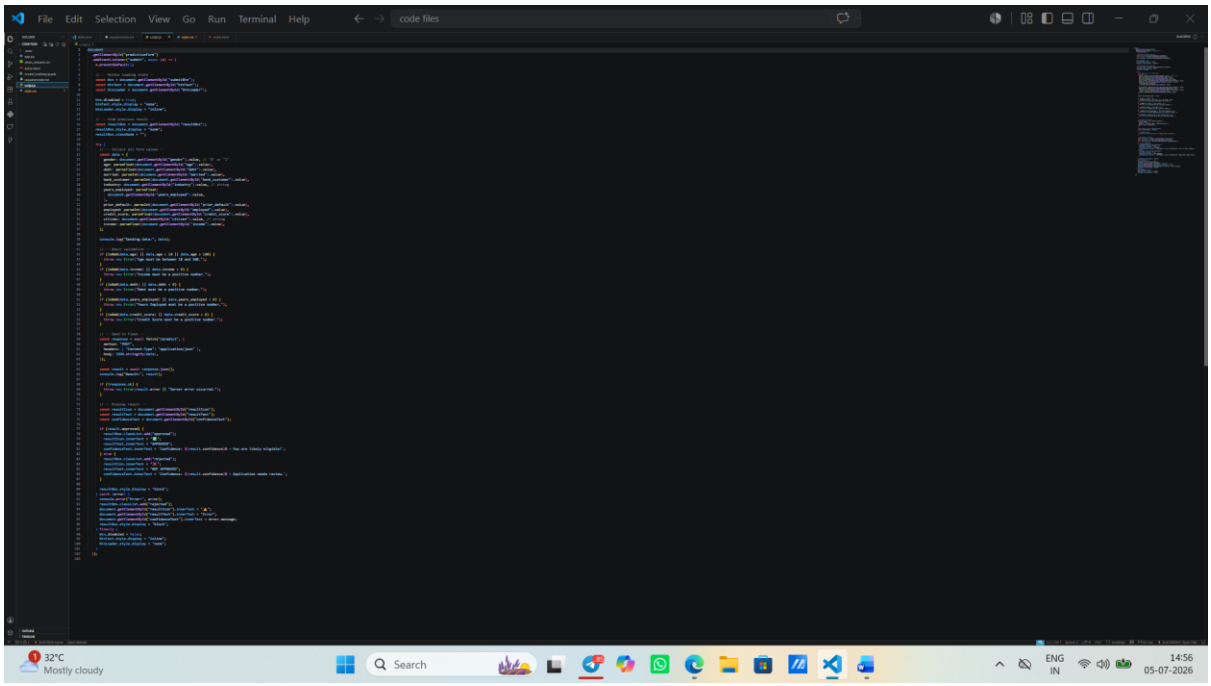
- HTML
- CSS
- JavaScript

HTML(structure)

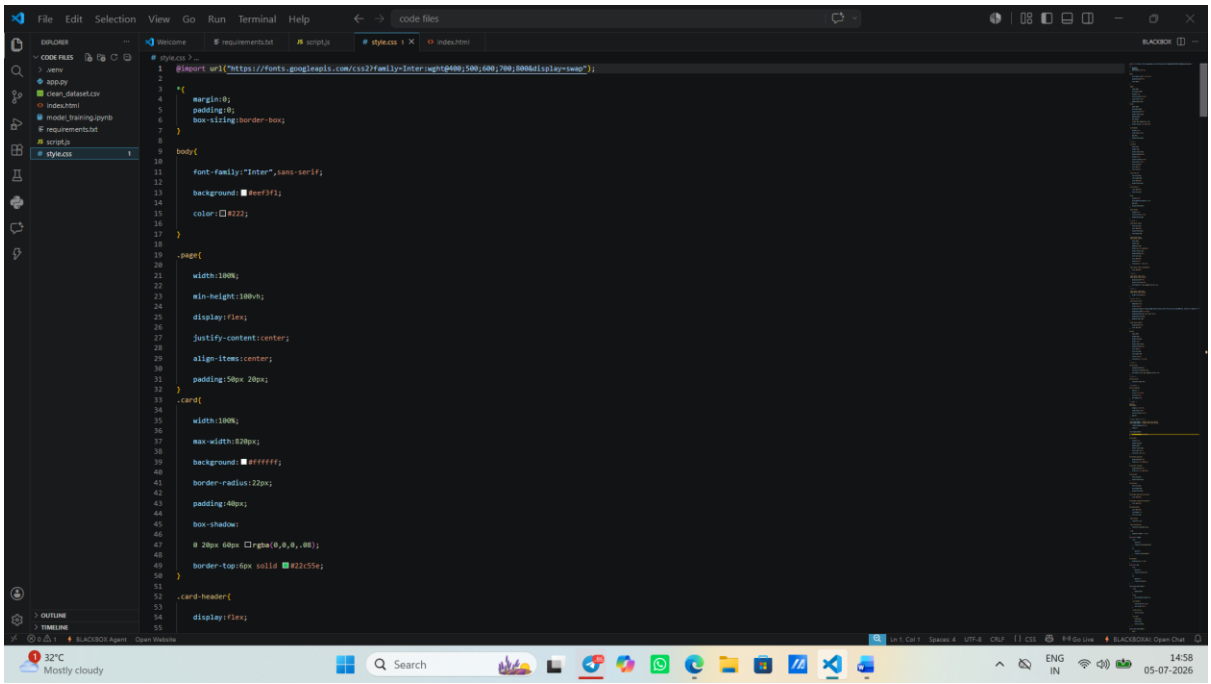




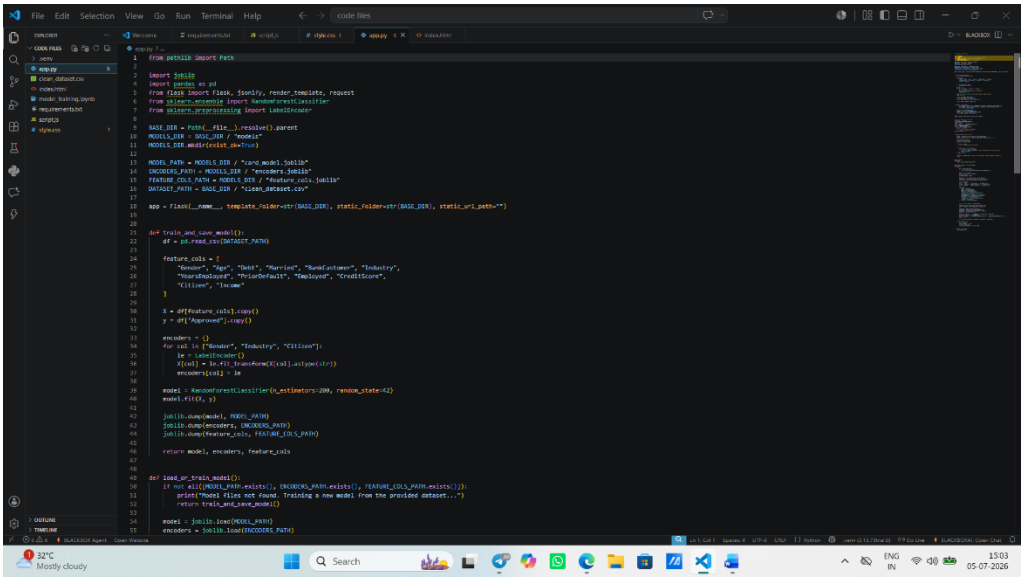
JavaScript



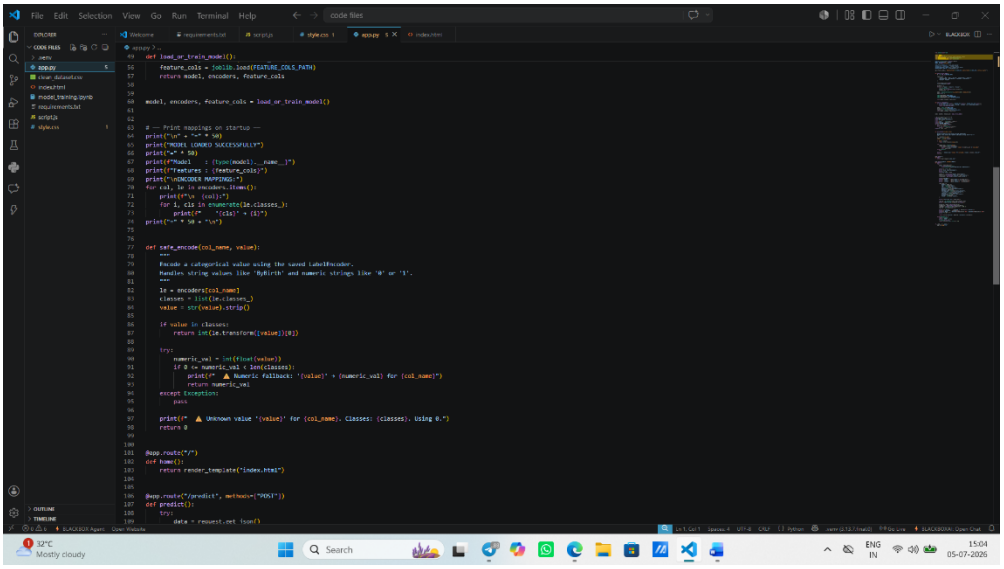
CSS



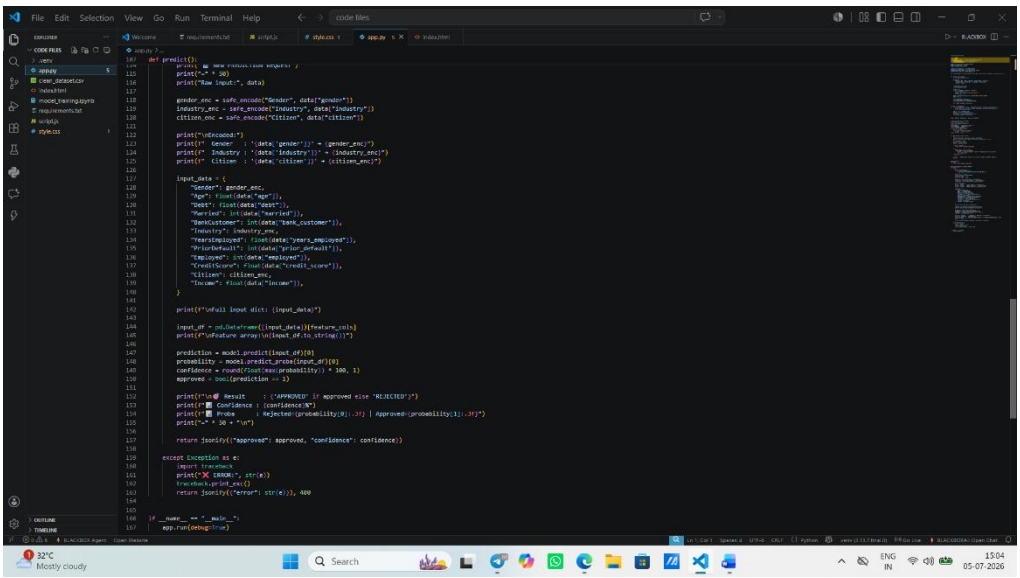
Backend Implementation Flask(app.py)



```python
from pathlib import Path

import joblib
import pandas as pd
from flask import Flask, jsonify, render_template, request
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder

BASE_DIR = Path(__file__).resolve().parent
MODELS_DIR = BASE_DIR / "models"
MODELS_DIR.mkdir(exist_ok=True)

MODEL_PATH = MODELS_DIR / "rand_model.joblib"
ENCODERS_PATH = MODELS_DIR / "encoders.joblib"
FEATURE_COLS_PATH = MODELS_DIR / "feature_cols.joblib"
DATASET_PATH = BASE_DIR / "clean_dataset.csv"

app = Flask(__name__, template_folder=str(BASE_DIR), static_folder=str(BASE_DIR), static_url_path="")


def train_and_save_model():
    df = pd.read_csv(DATASET_PATH)

    feature_cols = [
        "Gender", "Age", "Debt", "Married", "BankCustomer", "Industry",
        "YearsEmployed", "PriorDefault", "Employed", "CreditScore",
        "Citizen", "Income"
    ]

    X = df[feature_cols].copy()
    y = df["Approved"].copy()

    encoders = {}
    for col in ["Gender", "Industry", "Citizen"]:
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))
        encoders[col] = le

    model = RandomForestClassifier(n_estimators=200, random_state=42)
    model.fit(X, y)

    joblib.dump(model, MODEL_PATH)
    joblib.dump(encoders, ENCODERS_PATH)
    joblib.dump(feature_cols, FEATURE_COLS_PATH)

    return model, encoders, feature_cols


def load_or_train_model():
    if not all([MODEL_PATH.exists(), ENCODERS_PATH.exists(), FEATURE_COLS_PATH.exists()]):
        print("Model files not found. Training a new model from the provided dataset...")
        return train_and_save_model()

    model = joblib.load(MODEL_PATH)
    encoders = joblib.load(ENCODERS_PATH)
```



```python
def load_or_train_model():
    feature_cols = joblib.load(FEATURE_COLS_PATH)
    return model, encoders, feature_cols


model, encoders, feature_cols = load_or_train_model()


# -- Print mappings on startup --
print("\n" + "=" * 50)
print("MODEL LOADED SUCCESSFULLY")
print("=" * 50)
print(f"Model    : {type(model).__name__}")
print(f"Features : {len(feature_cols)}")
print("\nENCODER MAPPINGS:")
for col, le in encoders.items():
    print(f"\n  {col}:")
    for i, cls in enumerate(le.classes_):
        print(f"    '{cls}' -> {i}")
print("=" * 50 + "\n")


def safe_encode(col_name, value):
    """
    Encode a categorical value using the saved LabelEncoder.
    Handles string values like 'ByBirth' and numeric strings like '0' or '1'.
    """
    le = encoders[col_name]
    classes = list(le.classes_)
    value = str(value).strip()

    if value in classes:
        return int(le.transform([value])[0])

    try:
        numeric_val = int(float(value))
        if 0 <= numeric_val < len(classes):
            print(f"  ⚠ Numeric fallback: '{value}' -> {numeric_val} for {col_name}")
            return numeric_val
    except Exception:
        pass

    print(f"  ⚠ Unknown value '{value}' for {col_name}. Classes: {classes}. Using 0.")
    return 0


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():
    try:
        data = request.get_json()
```



```python
def predict():
        print("\n" + "=" * 50)
        print("Raw input:", data)

        gender_enc = safe_encode("Gender", data["gender"])
        industry_enc = safe_encode("Industry", data["industry"])
        citizen_enc = safe_encode("Citizen", data["citizen"])

        print("\nEncoded:")
        print(f"  Gender   : '{data['gender']}' -> {gender_enc}")
        print(f"  Industry : '{data['industry']}' -> {industry_enc}")
        print(f"  Citizen  : '{data['citizen']}' -> {citizen_enc}")

        input_data = {
            "Gender": gender_enc,
            "Age": float(data["age"]),
            "Debt": float(data["debt"]),
            "Married": int(data["married"]),
            "BankCustomer": int(data["bank_customer"]),
            "Industry": industry_enc,
            "YearsEmployed": float(data["years_employed"]),
            "PriorDefault": int(data["prior_default"]),
            "Employed": int(data["employed"]),
            "CreditScore": float(data["credit_score"]),
            "Citizen": citizen_enc,
            "Income": float(data["income"]),
            }

        print(f"\nFull input dict: {input_data}")

        input_df = pd.DataFrame([input_data])[feature_cols]
        print(f"\nFeature array:\n{input_df.to_string()}")

        prediction = model.predict(input_df)[0]
        probability = model.predict_proba(input_df)[0]
        confidence = round(float(max(probability)) * 100, 1)
        approved = bool(prediction == 1)

        print(f"\n🎯 Result     : {'APPROVED' if approved else 'REJECTED'}")
        print(f"📊 Confidence : {confidence}%")
        print(f"📊 Probs      : Rejected={probability[0]:.3f} | Approved={probability[1]:.3f}")
        print("=" * 50 + "\n")

        return jsonify({"approved": approved, "confidence": confidence})

    except Exception as e:
        import traceback
        print("❌ ERROR:", str(e))
        traceback.print_exc()
        return jsonify({"error": str(e)}), 400


if __name__ == "__main__":
    app.run(debug=True)
```

Conclusion and Future Scope

Conclusion

The Credit Card Approval Prediction using Machine Learning project successfully demonstrates how machine learning can be applied to automate and improve the credit card approval process. By analysing applicant information such as income, employment status, education, family status, housing type, and financial history, the developed model can accurately predict whether a credit card application is likely to be approved or rejected. Various classification algorithms were evaluated, and the best-performing model was integrated into a Flask-based web application to provide real-time predictions through a user-friendly interface. This system helps reduce manual effort, minimizes human bias, speeds up decision-making, and supports financial institutions in making consistent and reliable credit approval decisions. Overall, the project proves that machine learning is an effective solution for enhancing the efficiency, accuracy, and transparency of the credit card approval process.

Future Scope

The proposed Credit Card Approval Prediction system can be further enhanced with several advanced features to improve its performance and real-world usability. Future improvements include:

- Integrating larger and more diverse datasets to improve model accuracy and generalization.
- Deploying the application on cloud platforms such as AWS, Azure, or Google Cloud for better scalability and accessibility.
- Implementing deep learning techniques and ensemble learning methods to further enhance prediction performance.
- Adding Explainable AI (XAI) techniques such as SHAP or LIME to provide transparent explanations for each approval decision.
- Integrating real-time credit score and banking APIs to make predictions using live financial data.
- Incorporating fraud detection mechanisms to identify suspicious or fraudulent credit card applications.
- Developing a mobile application for easier access by customers and bank officials.
- Continuously retraining the model with new application data to adapt to changing customer behavior and financial trends.
- Implementing role-based authentication and enhanced security measures to protect sensitive customer information.
- Extending the system to support loan approval, insurance eligibility, and other financial risk assessment applications using the same machine learning framework.